Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Программирование графических процессоров»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 3. АТОМАРНЫЕ ОПЕРАЦИИ В CUDA

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является изучение принципов работы атомарных операций в CUDA, исследование их применения для обеспечения корректного параллельного доступа к памяти графического процессора, а также вычисление приближённого значения числа π методом Монте-Карло с использованием двумерной сетки нитей.

1.2. Задачи

- Изучить понятие атомарной операции и её отличие от обычных операций чтения и записи.
- Рассмотреть основные атомарные арифметические и побитовые операции CUDA.
- Изучить особенности атомарных операций в зависимости от области видимости и вычислительных возможностей устройства.
- Реализовать параллельное вычисление приближённого значения числа π методом Монте-Карло.
- Организовать подсчёт количества точек, попавших внутрь четверти единичного круга, с использованием atomicAdd.
- Проанализировать точность вычислений и роль атомарных операций в обеспечении корректности результата.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Понятие атомарных операций

Атомарная операция — это операция над данными в памяти, выполнение которой рассматривается как единое неделимое действие относительно других конкурирующих операций в пределах установленной области атомарности. В CUDA атомарные операции позволяют нескольким нитям безопасно изменять одну общую переменную без потери обновлений. В отличие от обычной последовательности чтения, изменения и записи, атомарная операция предотвращает ситуации, при которых несколько нитей одновременно считывают одно исходное значение и перезаписывают результаты друг друга. Большинство рассматриваемых операций возвращает старое значение, находившееся по указанному адресу до выполнения операции.

2.2. Основные атомарные арифметические операции

- atomicAdd — атомарно прибавляет заданное значение к числу по указанному адресу и возвращает его предыдущее значение.
- atomicSub — атомарно вычитает заданное значение из числа по указанному адресу.
- atomicExch — атомарно заменяет значение по адресу переданным значением и возвращает старое значение.
- atomicMin — записывает минимум из текущего и переданного значений.
- atomicMax — записывает максимум из текущего и переданного значений.
- atomicInc — сравнивает текущее значение с заданным пределом: если текущее значение больше предела, записывает ноль, иначе увеличивает его на единицу.
- atomicDec — если текущее значение равно нулю или превышает заданный предел, записывает предел; в противном случае уменьшает значение на единицу.
- atomicCAS — выполняет сравнение и обмен: записывает новое значение только в том случае, если текущее значение совпадает с ожидаемым.

2.3. Атомарные побитовые операции

CUDA поддерживает атомарные побитовые операции atomicAnd, atomicOr и atomicXor. Операция atomicAnd выполняет побитовое И, atomicOr — побитовое ИЛИ, а atomicXor — исключающее ИЛИ между текущим значением и переданным операндом. Результат записывается по исходному адресу, а функция возвращает значение, которое находилось там до выполнения операции. Эти функции применяются при обработке битовых масок, флагов состояния и других структур данных, в которых отдельные биты обозначают определённые признаки.

2.4. Области видимости и поддержка атомарных операций

Атомарные операции CUDA могут различаться по области видимости. Например, atomicAdd_block предназначена для атомарного взаимодействия между нитями одного блока, а atomicAdd_system обеспечивает атомарность относительно поддерживаемых операций со стороны CPU и GPU в пределах системы. Возможность применения конкретных вариантов зависит от архитектуры устройства, типа памяти и требований к синхронизации. Для некоторых типов данных и операций существуют дополнительные ограничения. При разработке программы необходимо учитывать вычислительную возможность устройства (Compute Capability), доступные типы данных и область атомарности. Атомарность сама по себе не заменяет все механизмы синхронизации и не гарантирует произвольный порядок выполнения нитей.

2.5. Метод Монте-Карло для вычисления числа π

Метод Монте-Карло позволяет приближённо вычислять число π с помощью случайных точек, равномерно распределённых в квадрате. Для единичного круга с центром в начале координат рассматривается четверть круга, расположенная в первой четверти координатной плоскости. Генерируются пары случайных чисел x и y из интервала [0, 1]. Точка считается попавшей внутрь четверти круга, если выполняется условие $x^2 + y^2 \leq 1$. Если N — общее число сгенерированных точек, а M — количество точек, попавших внутрь четверти круга, то приближённое значение числа π определяется формулой $\pi \approx 4M/N$. Чем больше число испытаний, тем, как правило, точнее статистическая оценка, однако результат каждого отдельного запуска может отличаться.

III. МЕТОДЫ И ПОДХОДЫ К РЕШЕНИЮ ЗАДАЧИ

3.1. Организация параллельных вычислений в CUDA

Вычисления выполняются на графическом процессоре с использованием CUDA-ядер. Для обработки N точек создаётся сетка блоков нитей. Индекс каждой нити определяется через blockIdx, blockDim и threadIdx. Например, для одномерной сетки глобальный индекс вычисляется выражением i = blockIdx.x * blockDim.x + threadIdx.x. Если индекс меньше общего числа точек, нить генерирует координаты очередной точки и проверяет принадлежность точки четверти единичного круга. Такой подход позволяет обрабатывать множество испытаний параллельно.

3.2. Генерация случайных координат

Для генерации координат следует реализовать device-функцию, доступную для вызова из CUDA-ядра. Она должна формировать псевдослучайные числа, равномерно распределённые на интервале [0, 1). Для получения независимых последовательностей каждой нити необходимо использовать подходящее состояние генератора случайных чисел, например Philox из библиотеки cuRAND, с уникальной последовательностью или идентификатором нити. Состояние генератора должно инициализироваться корректно, а повторное использование одного и того же состояния разными нитями следует исключить. Качество генерации случайных чисел непосредственно влияет на достоверность метода Монте-Карло.

3.3. Подсчёт точек с использованием atomicAdd

Для хранения количества точек, попавших внутрь четверти круга, используется общая целочисленная переменная. Перед запуском ядра она инициализируется нулём. Каждая нить проверяет условие x*x + y*y ≤ 1. Если условие выполняется, нить вызывает atomicAdd(result, 1). Атомарное сложение гарантирует, что одновременные обращения нескольких нитей не приведут к потере увеличений счётчика. После завершения выполнения ядра полученное значение используется для вычисления $\pi \approx 4M/N$.

3.4. Алгоритм выполнения практической работы

- Выбрать количество испытаний N и размер блока нитей.
- Выделить память для счётчика попаданий и инициализировать его нулём.
- Создать сетку блоков CUDA для обработки всех испытаний.
- В каждой нити сгенерировать случайные координаты x и y в диапазоне [0, 1).
- Проверить условие принадлежности точки четверти единичного круга: $x^2 + y^2 \leq 1$.
- При выполнении условия атомарно увеличить общий счётчик с помощью atomicAdd.
- Дождаться завершения ядра и получить значение счётчика на стороне CPU.
- Вычислить приближённое значение числа π по формуле 4M/N.
- Сравнить полученный результат с известным значением π и оценить абсолютную погрешность.

IV. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1. Принцип работы вычислительного ядра

В CUDA-ядре каждая нить получает уникальный глобальный индекс и обрабатывает одно испытание. По этому индексу или связанному с ним состоянию генератора создаются координаты случайной точки. Затем выполняется проверка x*x + y*y ≤ 1. Если точка принадлежит четверти круга, счётчик увеличивается атомарно. Нити, не удовлетворяющие условию, завершают обработку без изменения счётчика. После окончания работы всех нитей итоговое количество попаданий используется для оценки числа π.

4.2. Пример CUDA-ядра

```
#include

__global__ void initStates(curandStatePhilox4_32_10_t* states, unsigned long long seed, int n) {
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) {
curand_init(seed, i, 0, &states;[i]);
}
}

__global__ void monteCarloKernel(curandStatePhilox4_32_10_t* states, unsigned int* result, int n) {
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) {
curandStatePhilox4_32_10_t state = states[i];
float x = curand_uniform(&state;);
float y = curand_uniform(&state;);
states[i] = state;

if (x * x + y * y <= 1.0f) {
atomicAdd(result, 1u);
}
}
}

// Схема запуска на стороне CPU:
// 1. Выделить память для states и result.
// 2. Инициализировать result нулём.
// 3. Запустить initStates для N состояний генератора.
// 4. Запустить monteCarloKernel для N испытаний.
// 5. Скопировать result на CPU и вычислить pi = 4.0 * result / N.
// 6. Освободить выделенную память.
```

Примечание: curand_uniform возвращает значения из интервала (0, 1], что подходит для данной оценки площади. В полном приложении необходимо предусмотреть проверку ошибок CUDA, корректное выделение и освобождение памяти, синхронизацию перед чтением результата и подключение библиотеки cuRAND. Для использования приведённого фрагмента требуется передать массив состояний генератора, содержащий по одному состоянию на каждую обрабатываемую нить.

4.3. Расчёт приближённого значения числа π

После завершения вычислений обозначим количество испытаний через N, а число попаданий в четверть круга — через M. Приближённое значение числа π рассчитывается как π_прибл = 4M/N. Для оценки точности используется абсолютная погрешность ε = |π_прибл − π|, где эталонное значение π приблизительно равно 3,141592653589793. Например, если при N = 1000000 испытаний получено M = 785398 попаданий, то π_прибл = 4 × 785398 / 1000000 = 3,141592. Этот пример иллюстрирует вычисление результата, но не является гарантированным результатом реального запуска программы.

4.4. Анализ точности и производительности

- Увеличение количества испытаний обычно улучшает статистическую оценку числа π, однако не гарантирует монотонного уменьшения ошибки при каждом запуске.
- Для метода Монте-Карло характерна типичная скорость уменьшения статистической погрешности порядка 1/√N.
- Использование atomicAdd обеспечивает корректный подсчёт попаданий при параллельном выполнении нитей.
- При большом количестве попаданий обращение к единому глобальному счётчику может стать узким местом производительности из-за конкуренции за атомарное обновление.
- Для ускорения вычислений можно использовать локальные счётчики на уровне блока и затем объединять частичные результаты, сокращая количество обращений к глобальному счётчику.
- На результат и скорость выполнения влияют размер блока, количество испытаний, архитектура GPU, способ генерации случайных чисел и организация доступа к памяти.

4.5. Контроль корректности результатов

- Перед запуском вычислительного ядра счётчик попаданий должен быть равен нулю.
- Количество обработанных испытаний должно совпадать с заданным N.
- Итоговое количество попаданий должно находиться в диапазоне от нуля до N включительно.
- Приближённое значение числа π должно вычисляться с использованием отношения 4M/N.
- Для проверки корректности следует провести несколько запусков и сравнить результаты с эталонным значением π.
- При измерении производительности необходимо учитывать время инициализации генератора, выполнение ядра и передачу данных, если эти этапы входят в измеряемый интервал.

V. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

5.1. Что такое атомарная операция? В чём её ключевое отличие от обычных операций?

Атомарная операция — это неделимое относительно конкурирующих обращений действие над общей переменной в установленной области атомарности. Её ключевое отличие от обычной последовательности операций заключается в том, что другие нити не могут вмешаться в атомарное обновление так, чтобы нарушить его целостность. Например, если несколько нитей одновременно увеличивают общий счётчик, обычная последовательность чтения, увеличения и записи может привести к потере обновлений. Использование atomicAdd позволяет корректно учитывать каждое увеличение.

5.2. Какие атомарные операции существуют в CUDA?

К основным атомарным операциям CUDA относятся арифметические операции atomicAdd, atomicSub, atomicExch, atomicMin, atomicMax, atomicInc и atomicDec; операция сравнения и обмена atomicCAS; а также побитовые операции atomicAnd, atomicOr и atomicXor. В зависимости от версии CUDA, архитектуры GPU, типа данных и области видимости могут быть доступны дополнительные варианты, включая операции с суффиксами _block и _system. Конкретная поддержка определяется документацией CUDA для используемого устройства.

VI. ВЫВОДЫ

В ходе практической работы рассматриваются принципы использования атомарных операций CUDA для организации корректного доступа нескольких нитей к общей памяти. Изучены основные арифметические и побитовые атомарные операции, а также особенности области их видимости. Для решения практической задачи применяется метод Монте-Карло: случайные точки генерируются в квадрате [0, 1] × [0, 1], после чего определяется количество точек, попавших в четверть единичного круга. Атомарная операция atomicAdd позволяет безопасно накапливать число попаданий при параллельном выполнении вычислений. Приближённое значение числа π рассчитывается по формуле $\pi \approx 4M/N$, а точность оценивается сравнением с эталонным значением. Таким образом, практическая работа демонстрирует применение атомарных операций для решения задачи параллельного подсчёта и подчёркивает важность корректной генерации случайных чисел, выбора конфигурации CUDA и анализа статистической погрешности.